

Transport Layer: UDP

Henry Quispe, *UTEC*, Renato Garcia, *UTEC* César Carranza, *UTEC*, Sofia Ku, *UTEC*, Ariann Salcedo, *UTEC*,

I. INTRODUCTION

UDP es un protocolo de transporte que permite enviar mensajes independientes sin establecer previamente una conexión entre emisor y receptor [1]. Su header reducido y la ausencia de confirmaciones propias del protocolo lo hacen útil para estudiar cómo se intercambian datagramas y qué ocurre cuando la aplicación debe gestionar pérdidas o errores. En este laboratorio se analizaron capturas de Wireshark correspondientes a tráfico UDP, consultas DNS y mensajes enviados mediante programas en Python y Netcat. También se realizaron pruebas de comunicación entre equipos, difusión en una red local y envío intensivo de datagramas. A partir de las capturas se examinan los campos de los paquetes, su recorrido y las diferencias observables frente a TCP. Finalmente, se relacionan los resultados con las limitaciones de UDP, en particular la ausencia de control de flujo y de confirmación de entrega.

A. Theoretical Framework

1) *UDP*: User Datagram Protocol (UDP) es un protocolo de transporte que envía datagramas independientes sin establecer previamente una conexión. Su encabezado contiene cuatro campos de 16 bits: puerto de origen, puerto de destino, longitud y checksum [2]. UDP no incorpora confirmaciones de entrega, retransmisiones ni control de flujo. Si una aplicación requiere recuperación o detección de desconexiones, debe programar mecanismos como confirmaciones, temporizadores o mensajes periódicos [1], [2].

2) *TCP*: Transmission Control Protocol (TCP) es un protocolo de transporte orientado a conexión que proporciona una entrega confiable y ordenada de los datos [1], [3]. A diferencia de UDP, establece una conexión mediante un *three-way handshake* y utiliza números de secuencia, confirmaciones ACK y retransmisiones para gestionar la entrega de los datos [3].

3) *DNS*: Domain Name System (DNS) es un sistema jerárquico y distribuido que traduce nombres de dominio en direcciones IP y almacena otros datos asociados a los servicios de red [4]. Sus consultas y respuestas pueden transportarse mediante UDP usando el puerto 53 del servidor. DNS admite tanto TCP como UDP [5].

4) *TCP Window Size and Flow Control*: En TCP, el receptor anuncia cuánto espacio tiene disponible, Window Size, mediante la ventana de recepción. El emisor utiliza esa información para limitar la cantidad de datos pendientes de confirmación y evitar que el receptor reciba más de lo que puede almacenar. Este es un mecanismo de TCP utilizados para evitar que el receptor descarte paquetes por un desbordamiento de búfer (buffer overflow). UDP no posee una ventana de recepción equivalente [2], [3].

5) *Datagram Length and Fragmentation*: La longitud indicada en el encabezado UDP incluye sus 8 bytes de encabezado y los datos transportados [2]. UDP no posee una ventana de recepción equivalente a la de TCP. Si un paquete IP supera la unidad de transmisión máxima (MTU) del trayecto, este puede requerir fragmentación. La pérdida de uno de los fragmentos impide reconstruir el datagrama completo [6].

6) *Checksum*: El checksum es un campo que permite detectar determinadas inconsistencias en un datagrama recibido. El emisor aplica un algoritmo matemático para obtener un valor que se coloca en el encabezado, y el receptor puede volver a calcular el valor para comprobar si coincide [1], [2]. En herramientas de captura como Wireshark, una etiqueta *unverified* indica que Wireshark no verificó el checksum. El *checksum offloading* es una posible explicación, pero la etiqueta no demuestra por sí sola que se haya utilizado ni que el paquete esté corrupto [7].

7) *TTL*: Time to Live (TTL) es un valor en el encabezado del IPv4 que indica el número máximo de saltos que un paquete puede realizar en la red. Por cada salto a través de un router, el valor disminuye en uno; si llega a cero, el router descarta el paquete para evitar que la información circule de forma infinita [1].

8) *ICMP*: Internet Control Message Protocol (ICMP) comunica determinadas condiciones y errores de la capa de red [1], [8]. Cuando un datagrama UDP se envía a un puerto cerrado el receptor puede responder con un mensaje ICMPv4 *Destination Unreachable* con tipo 3 y código 3 indica *Port Unreachable*. El mensaje incluye el encabezado IP original y al menos los primeros bytes de sus datos, lo que permite relacionar el error con el datagrama que provocó el fallo [8].

9) *Unicast, Loopback and Broadcast*: En una transmisión *unicast*, el paquete se dirige a un destinatario específico. La dirección 127.0.0.1 permite probar la comunicación loopback dentro del mismo equipo [1]. En cambio, una dirección IPv4 de broadcast permite dirigir un paquete a los equipos de una red local que admite difusión. En Ethernet, la dirección MAC de broadcast es ff:ff:ff:ff:ff:ff [1], [9]. UDP permite enviar un datagrama a una dirección de broadcast; TCP establece conexiones entre extremos específicos y no ofrece ese tipo de transmisión [2], [3].

10) *DHCP*: Dynamic Host Configuration Protocol (DHCP) permite que un equipo obtenga una dirección IP y otros parámetros de configuración. Durante el descubrimiento inicial, el cliente puede enviar un mensaje UDP por broadcast porque todavía no conoce la dirección del servidor DHCP [10].

11) *UDP Flood Attack*: Un ataque UDP flood consiste en enviar un gran volumen de datagramas UDP hacia un destino con el propósito de sobrecargar la red o los recursos de procesamiento del receptor [11].

12) *DoS and DDoS*: Un ataque de denegación de servicio (DoS) busca impedir que un servicio atienda normalmente a sus usuarios al sobrecargar sus recursos. Cuando el tráfico proviene de múltiples equipos, se denomina ataque distribuido de denegación de servicio (DDoS) [1].

13) *Netcat*: Netcat es una herramienta de línea de comandos que permite enviar y recibir datos mediante protocolos como TCP y UDP. La variante Ncat incluye modos de escucha y envío; al seleccionar UDP, puede enviar datos sin establecer previamente una conexión TCP [2], [12].

14) *Wireshark*: Herramienta de código abierto empleada para capturar y analizar el tráfico que circula por una interfaz de red [7]. Permite inspeccionar detalladamente la estructura de los paquetes UDP y TCP, verificar puertos, examinar direcciones MAC e IP, e identificar mensajes de error o secuencias de carga.

B. State of the Art

El estudio de los protocolos de transporte ya no se limita a comparar UDP y TCP de forma aislada. Actualmente también se analiza cómo las aplicaciones utilizan estos protocolos y qué ocurre cuando cambian las condiciones de la red, aumenta el tamaño de los mensajes o se concentra mucho tráfico en un receptor. Estos temas aparecen tanto en servicios cotidianos, como DNS y la navegación web, como en investigaciones sobre la confiabilidad y la seguridad de las comunicaciones.

En el caso de la web, Trevisan et al. [?] estudiaron la adopción y el rendimiento de HTTP/3, que utiliza QUIC sobre UDP. Encontraron mejoras principalmente en redes con alta latencia o poco ancho de banda. Sin embargo, QUIC incorpora sus propios mecanismos de confiabilidad y control, por lo que este resultado no puede atribuirse únicamente al uso de UDP. El trabajo muestra que la elección del protocolo de transporte debe evaluarse junto con las funciones que implementa la aplicación y las condiciones en las que se comunica.

La relación entre el tamaño de los mensajes y el transporte también se ha estudiado en DNS. Kosek et al. [?] compararon consultas DNS sobre UDP y TCP realizadas desde distintos puntos de la red. Observaron que TCP generalmente tenía un mayor tiempo de respuesta, pero podía ser necesario para obtener respuestas que resultaban demasiado grandes para el intercambio previsto mediante UDP. En la misma línea, Bonica et al. [6] explican que la fragmentación IP puede hacer menos confiable la entrega de paquetes grandes, ya que la pérdida de un fragmento impide reconstruir el paquete completo. Por ello, el tamaño de un mensaje no solo influye en cómo se envía, sino también en la posibilidad de recibirlo correctamente.

Además del tamaño, la cantidad de datagramas enviados en poco tiempo puede afectar al receptor. Gelenbe y Nasereddin [11] estudiaron la congestión producida durante un ataque UDP flood contra un servidor y evaluaron un mecanismo para regular el tráfico entrante. Sus resultados muestran que el problema no es solo el ancho de banda disponible, el servidor también necesita recursos para recibir, procesar y distinguir los paquetes legítimos de los que forman parte del ataque.

II. DEVELOPMENT

A. Checkpoint 1: Simple UDP Capture

1) How many UDP packets were captured during your session? Were they mostly sent or received by your host?

En la captura filtrada se observan 19 paquetes UDP asociados con nuestro equipo, 172.20.10.7: 9 enviados y 10 recibidos (Fig. 1). Por tanto, se recibieron ligeramente más paquetes de los que se enviaron. Al filtrar la dirección 34.139.173.2, asociada con la página de UTEC en esta prueba, el tráfico mostrado corresponde a a TCP y TLS, no a UDP (Fig. 2).

No.	Time	Source	Destination	Protocol	Length	Info
177.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
178.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
179.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
180.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
181.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
182.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
183.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
184.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
185.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
186.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
187.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
188.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
189.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
190.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
191.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
192.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
193.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
194.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
195.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
196.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
197.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]

Fig. 1. Paquetes UDP enviados y recibidos durante la captura.

No.	Time	Source	Destination	Protocol	Length	Info
177.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
178.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
179.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
180.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
181.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
182.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
183.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
184.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
185.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
186.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
187.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
188.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
189.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
190.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
191.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
192.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
193.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
194.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
195.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000
196.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	58612 → 443 [len=3] Seq=1000000000
197.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	TCP	60	443 → 58612 [len=3] Seq=1000000000

Fig. 2. Tráfico TCP y TLS registrado hacia la dirección IP de UTEC.

2) Did you notice any UDP traffic that was not related to accessing the www.utec.edu.pe website? If so, what could be the source of that traffic?

Sí, durante la sesión si hubo tráfico UDP que no estaba relacionado al acceso a www.utec.edu.pe (Fig. 3). Las consultas DNS podrían proceder del navegador o de procesos en segundo plano.

No.	Time	Source	Destination	Protocol	Length	Info
177.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
178.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
179.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
180.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
181.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
182.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
183.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
184.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
185.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
186.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
187.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
188.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
189.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
190.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
191.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
192.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
193.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
194.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
195.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]
196.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	58612 → 443 [len=3]
197.9.58612200	21.195.99.211	172.20.10.7	34.139.173.2	UDP	72	443 → 58612 [len=3]

Fig. 3. Paquetes UDP con destinos distintos a la página de UTEC.

3) Pick a UDP packet from your host and describe its main fields (e.g., source port, destination port, length, checksum).

Campos principales de un paquete UDP (Fig. 4) :

- Source port: 62068
- Destination port: 443

167	4.387984100	10.100.210.85	142.250.0.95	UDP	1094	62068	→ 443	Len=1052
169	4.407331000	142.250.0.95	10.100.210.85	UDP	69	443	→ 62068	Len=27
171	4.424687300	142.250.0.95	10.100.210.85	UDP	1078	443	→ 62068	Len=1036
172	4.424803600	142.250.0.95	10.100.210.85	UDP	69	443	→ 62068	Len=27
173	4.424804800	142.250.0.95	10.100.210.85	UDP	239	443	→ 62068	Len=197
174	4.425246000	10.100.210.85	142.250.0.95	UDP	77	62068	→ 443	Len=35
175	4.425454100	10.100.210.85	142.250.0.95	UDP	73	62068	→ 443	Len=31
176	4.426729400	142.250.0.95	10.100.210.85	UDP	69	443	→ 62068	Len=27
177	4.432611600	64.233.186.95	10.100.210.85	UDP	68	443	→ 54976	Len=26
178	4.445070300	142.250.0.95	10.100.210.85	UDP	1013	443	→ 62068	Len=971
179	4.445071800	142.250.0.95	10.100.210.85	UDP	214	443	→ 62068	Len=172
180	4.445537500	10.100.210.85	142.250.0.95	UDP	77	62068	→ 443	Len=35
181	4.446180300	142.250.0.95	10.100.210.85	UDP	1010	443	→ 62068	Len=968
182	4.446189200	142.250.0.95	10.100.210.85	UDP	192	443	→ 62068	Len=150
183	4.446190300	142.250.0.95	10.100.210.85	UDP	1028	443	→ 62068	Len=986


```

Frame 167: Packet, 1094 bytes on wire (8752 bits), 1094 bytes captured (8752 bits) on interface \Device\NPF...
Ethernet II, Src: LiteonTechno... (08:1e:a4:a9:46:a9), Dst: Cisco_ff:ff:20 (00:08:e3:ff:ff:20)
Internet Protocol Version 4, Src: 10.100.210.85, Dst: 142.250.0.95
User Datagram Protocol, Src Port: 62068, Dst Port: 443
  Source Port: 62068
  Destination Port: 443
  Length: 1060
  Checksum: 0x4625 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 16]
  [Stream Packet Number: 9]
  [Timestamps]
    UDP payload (1052 bytes)
    Data (1052 bytes)

```

Fig. 4. Puertos, longitud y checksum del datagrama UDP seleccionado.

- Length: 1060 bytes
- Checksum: 0x4625 [unverified]

4) **Is the UDP checksum marked as verified or unverified? What does this indicate in the context of your operating system and network interface?**

Está marcado como unverified (Fig. 4). Unverified checksum indica que el cálculo del checksum no fue verificado por Wireshark, esto posiblemente se da porque el cálculo fue delegado a la tarjeta de red.

5) **Compare UDP with TCP based on what you observed in Wireshark. What are the key differences in terms of packet structure and reliability?**

En la captura de Wireshark realizada, se puede comprobar la alta confiabilidad de TCP a través de sus banderas. Observamos paquetes con la etiqueta [ACK] lo que demuestra el mecanismo de reporte de saludo que utiliza TCP para confirmar que los datos llegaron a su destino. También se aprecian banderas [PSH], utilizadas para forzar la entrega inmediata de datos a la aplicación (Fig. 6). En contraste, los paquetes UDP no poseen estas banderas de control ni realizan confirmaciones, enviando los datos sin garantizar su recepción (Fig. 5).

```

Frame 90: Packet, 71 bytes on wire (568 bits), 71 bytes captured (568 bits) on interface \Device\NPF...
Ethernet II, Src: Intel_f4:5a:8d (c4:bd:e5:f4:5a:8d), Dst: 0e:e2:32:13:ae:19 (0e:e2:32:13:ae:19)
Internet Protocol Version 4, Src: 10.31.227.65, Dst: 142.250.0.95
User Datagram Protocol, Src Port: 57387, Dst Port: 443
  Source Port: 57387
  Destination Port: 443
  Length: 37
  Checksum: 0x7cf0 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 0]
  [Stream Packet Number: 3]
  [Timestamps]
    UDP payload (29 bytes)
    Data (29 bytes)
      Dates: 57b6d91c351b9221b1c7697a6d12041df0717015b0a7326fcb0c044b6
      [Length: 29]

```

Fig. 5. Campos del datagrama UDP utilizado para la comparación.

6) **What are the potential risks or vulnerabilities of using UDP in a network? Did you observe any unusual or potentially malicious UDP traffic?**

Como UDP no confirma ni retransmite los datagramas, algunos riesgos o vulnerabilidades de usar UDP en un network es que los paquetes enviados se pueden perderse, duplicarse o llegar desordenados. Así mismo,

```

Frame 94: Packet, 278 bytes on wire (2224 bits), 278 bytes captured (2224 bits) on interface \Device\NPF...
Ethernet II, Src: 0e:e2:32:13:ae:19 (0e:e2:32:13:ae:19), Dst: Intel_f4:5a:8d (c4:bd:e5:f4:5a:8d)
Internet Protocol Version 4, Src: 157.240.197.60, Dst: 10.31.227.65
Transmission Control Protocol, Src Port: 5222, Dst Port: 50418, Seq: 136, Ack: 75, Len: 224
  Source Port: 5222
  Destination Port: 50418
  [Stream index: 2]
  [Stream Packet Number: 5]
  [Conversation completeness: Incomplete (12)]
  [TCP Segment Len: 224]
  Sequence Number: 136 (relative sequence number)
  Sequence Number (raw): 1194055726
  [Next Sequence Number: 360 (relative sequence number)]
  Acknowledgment Number: 75 (relative ack number)
  Acknowledgment number (raw): 1627556026
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x018 (PSH, ACK)
  Window: 2030
  [Calculated window size: 2030]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0x214d [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]

```

Fig. 6. Campos y banderas del segmento TCP utilizado para la comparación.

este facilita la falsificación de direcciones IP y ataques DDos donde pueden saturar un servidor o red [2], [11]. Durante la captura no se observó tráfico UDP inusual o potencialmente malicioso. Los paquetes que se registraron parecen ser de servicios normales ejecutados en segundo plano por el navegador o el sistema (Fig. 1).

7) **How could an administrator use Wireshark to detect suspicious UDP behavior, such as a port scan or DDos attempt?**

Un administrador puede utilizar Wireshark para monitorear el tráfico UDP y detectar anomalías analizando los siguientes indicadores [7]:

- **Detección de Port Scanning:** Al aplicar un filtro de tráfico UDP y revisar el panel Statistics ¿ Conversations, el administrador puede identificar si una misma dirección IP de origen intenta establecer comunicación con múltiples puertos de un host destino en un lapso corto de tiempo. Un patrón recurrente de mensajes ICMP *Destination Unreachable (Port Unreachable)* devueltos también suele ser indicativo de un escaneo activo.
- **Detección de DDos:** Utilizando Statistics ¿ I/O Graphs, se puede visualizar un aumento abrupto en la densidad de paquetes por segundo (pkts/sec) hacia un equipo o red específica. Al cruzar este pico con el panel de conversaciones, se observará un tráfico masivo desde una o múltiples fuentes que contrasta drásticamente con la línea base de uso habitual.

B. Checkpoint 2: Using nslookup

1) **How many UDP packets were captured during the nslookup www.cisco.com query? Were they sent, received, or both?**

En la lista se observan cuatro paquetes DNS asociados con las consultas de www.cisco.com: dos solicitudes enviadas por el equipo y dos respuestas recibidas (Fig. 7 y Fig. 8).

2) **What are the source and destination port numbers of the captured DNS packets? What does this indicate about how DNS communication works?**

En una de las solicitudes se observa el puerto de origen 58803 y el puerto de destino 53 (Fig. ??). El equipo

Interface 4	IPv4	IPv6	TCP	UDP	
Direction	Porto A	Direction B	Porto B	Packets	Bytes
10.100.243.118	58	255.255.255.255	67	4	148
10.100.243.118	58804	10.100.243.118	53	4	178 bytes
10.100.243.118	58804	10.100.243.118	53	2	168 bytes
10.100.243.118	58804	10.100.243.118	53	2	174 bytes
10.100.243.118	58804	10.100.243.118	53	2	174 bytes
10.100.243.118	54165	142.251.6.113	443	2	174 bytes
172.17.179.25	443	10.100.243.118	50242	2	470 bytes

Fig. 7. Conversaciones UDP registradas durante la consulta DNS.

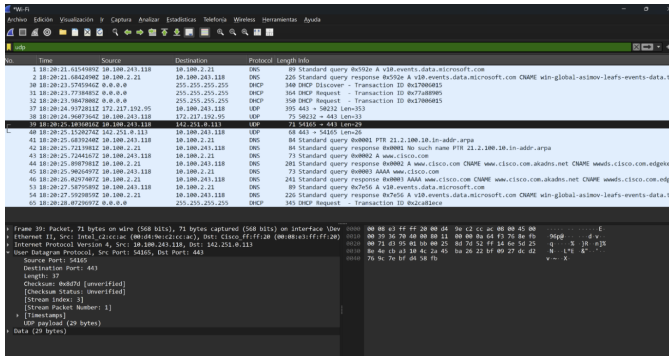


Fig. 8. Solicitudes y respuestas DNS observadas en Wireshark.

utiliza un puerto temporal y dirige la consulta al puerto DNS del servidor. En la respuesta correspondiente, el servidor utiliza el puerto 53 como origen y responde al puerto temporal de esa solicitud [5].

- 3) Describe the typical UDP length for the DNS request and response observed. Are the packet sizes similar or different? Why might that be?

Una de las solicitudes tiene una longitud UDP de 73 bytes, y las respuestas son mayores, de 201 y 241 bytes (Fig. ??). Las respuestas son más largas porque tienen información adicional como direcciones IP y records CNAME.

- 4) Was the DNS query resolved successfully? How can you tell this by analyzing the UDP response in Wireshark?

Sí, la DNS query respondió correctamente. Lo podemos identificar ya que la respuesta seleccionada tuvo en su flags tiene la etiqueta *Standard query response, No error* y tiene registros normales de respuesta (Fig. 9).

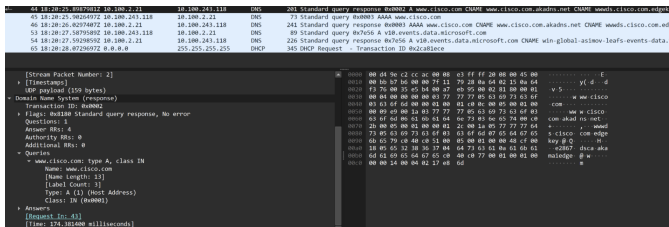


Fig. 9. Respuesta DNS de www.cisco.com sin errores y con registros en Answers.

- 5) Why is DNS usually implemented over UDP instead of TCP? Based on your observations, what advantages does UDP offer in this case?

Los DNS usualmente utilizan UDP en vez de TCP porque UDP permite enviar una consulta DNS y recibir su respuesta sin establecer antes una conexión TCP. Esto evita el intercambio inicial necesario para abrir esa

conexión y resulta una ventaja al ser más práctico para consultas cortas.

- 6) Expand the UDP layer of a selected DNS packet. What are the values of the source port, destination port, length, and checksum? Provide a brief explanation of each field

En el paquete seleccionado, el Source Port es 53, el Destination Port es 58804, Length es de 167 bytes y el Checksum es 0xeb95 (Fig. 10). El source y destination port identifican el envío y el recibo; el length especifica el tamaño del UDP header y payload, y el checksum se usa para detectar errores en el datagrama [2].

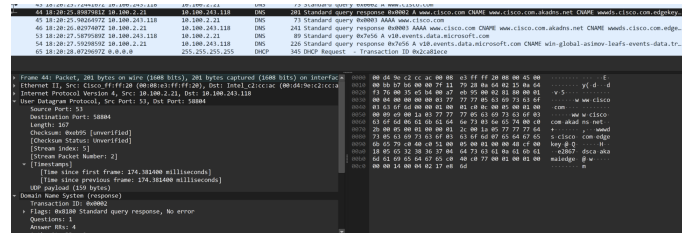


Fig. 10. Puertos, longitud y checksum de una respuesta DNS transportada por UDP.

- 7) What is the maximum number of bytes that can be included in a UDP payload? (Research and present in the lab session)

En IPv4, con un encabezado IP mínimo de 20 bytes, el máximo teórico de datos de un datagrama UDP es de 65 507 bytes. Se obtiene al restar al máximo de 65 535 bytes del paquete IPv4 los 20 bytes de su encabezado y los 8 bytes del encabezado UDP: $65\,535 - 20 - 8 = 65\,507$ [1], [2]. Ese máximo teórico no garantiza que un datagrama de ese tamaño pueda enviarse o recibirse completo por cualquier trayecto [6].

- 8) What is the largest possible source port number? (Research and present in the lab session).

Los puertos usan 16 bits, por lo que el mayor número que puede representar el campo de puerto de origen es $2^{16} - 1 = 65\,535$ [2].

- 9) What is the protocol number for UDP? Give your answer in both hexadecimal and decimal notation. To answer this question, you'll need to look into the Protocol field of the IP datagram containing this UDP segment.

En el campo Protocol del encabezado IP se observa el valor 17 en decimal, equivalente a 0x11 en hexadecimal (Fig. 11). Este valor identifica a UDP como el protocolo transportado por IP.

- 10) Examine a pair of UDP packets in which your host sends the first UDP packet and the second UDP packet is a reply to this first UDP packet. Describe the relationship between the port numbers in the two packets.

En la consulta DNS, nuestro equipo envía el paquete desde puerto 54165 hacia el puerto 53. En la respuesta, los puertos aparecen invertidos: 53 es el origen y el mismo puerto 54165 pasa a ser el destino (Fig. ?? y

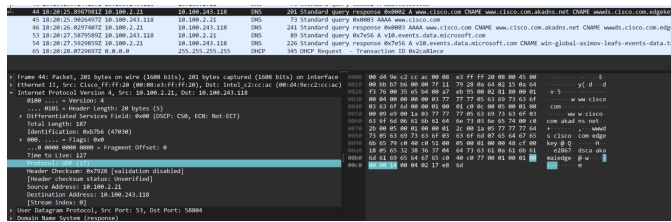


Fig. 11. Valor del campo Protocol en el encabezado IPv4 de un paquete UDP.

(Fig. ??).

C. Checkpoint3: Python UDP Implementation (Loopback)

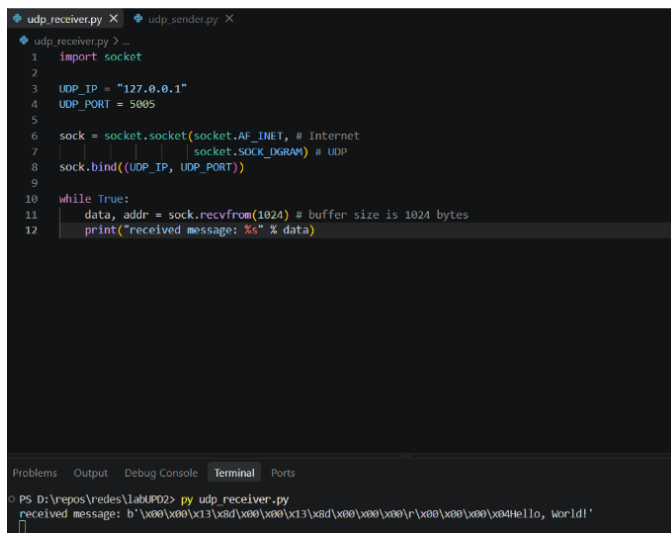


Fig. 12. Código y salida del receptor UDP configurado en 127.0.0.1:5005.

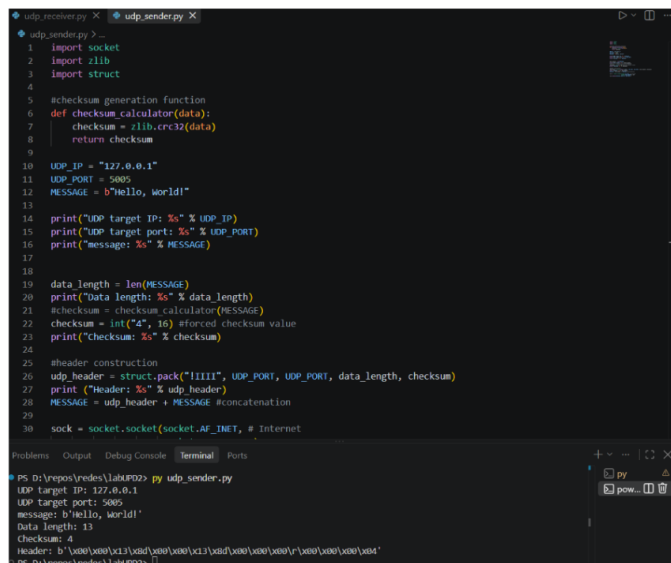


Fig. 13. Código y salida del emisor UDP configurado para enviar al puerto 5005.

- 1) Does the received header transmitted the data defined in our python program?

Sí, el header recibido si transmite la data definida por el programa de python. Esto se observa en el mensaje impreso en el sender indica el header

b'\\x00\\x00\\x13\\x8d\\x00\\x00\\x13\\x8d\\x00\\x00\\x00\\x1\\x00\\x00\\x04\\Hello, world!'

es el mismo que se observa en el mensaje que recibió el receiver.(Fig. 12)

- 2) **What's the UDP IP on the code.** La UDP IP definida en el código es 127.0.0.1. (Fig. 12)
- 3) **Analyze the UDP package and explain the UDP datagram.**

El origen de este datagrama sería el 49825 y lo está enviando al 5005. Cuenta con un tamaño de 37 y el Checksum es 0xc31b [unverified]. El unverified significa que el wireshark no ha aplicado el algoritmo matemático para verificar si es que la data se corrompió o si se pierde información durante el envío. (Fig. 14)

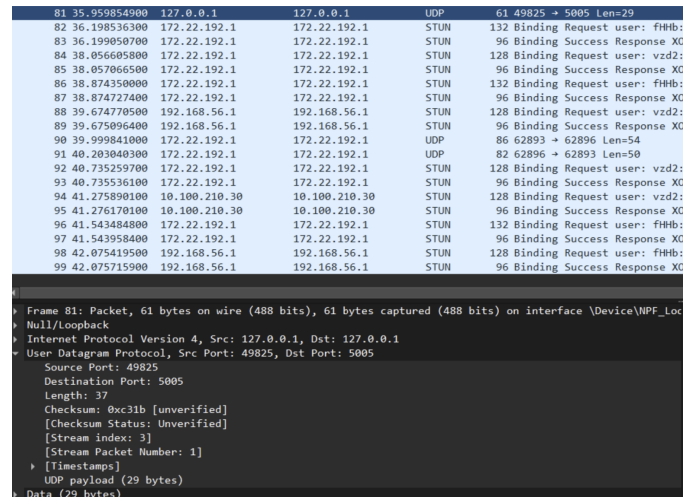


Fig. 14. Datagrama UDP enviado por loopback al puerto 5005.

- 4) **Answer, what would you need to do to send the UDP packet from one computer to a different one? Is any material required? How about the IP configuration?** Para poder mandar un paquete UDP de una computadora a otra se necesitaría que las computadoras estén conectadas a la misma red para poder asignarles direcciones IP que pertenezcan a la misma subred. No se necesita ningún material adicional. En cuanto a la configuración de la IP, en el código del emisor se debe poner la dirección IP y el puerto UDP del reciever, mientras que el receptor debe tener abierto un puerto UDP para poder escuchar en el puerto que se envía el paquete.
- 5) **Compare the UDP checksum field in at least two captured packets. Was the checksum calculated and validated correctly? Explain how you verified this.**

No, como nos dice que el status del Checksum es unverified, significa que el wireshark decidió no realizar los cálculos para verificar el estado de la información recibida y comprarla con la enviada. La manera en la que se debería verificar es que el emisor aplica un algoritmo matemático en el cuál obtiene un número que se coloca

en el header del emisor. Luego al recibir la información el receptor, vuelve a ejecutar el mismo algoritmo para ver si el número que obtiene tras el algoritmo coincide con el mismo que obtuvo el emisor. Si los números no coinciden, significa que la información se corrompe o se perdió parte de esta. (Fig. 15 y Fig. 16)

81	35.959854900	127.0.0.1	127.0.0.1	UDP	61 49825 → 5005 Len=29
82	36.198536300	172.22.192.1	172.22.192.1	STUN	132 Binding Request user: fHhb:
83	36.199050700	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
84	38.056605800	172.22.192.1	172.22.192.1	STUN	128 Binding Request user: vvd2:
85	38.057066500	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
86	38.874350000	172.22.192.1	172.22.192.1	STUN	132 Binding Request user: fHhb:
87	38.874727400	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
88	39.674770500	192.168.56.1	192.168.56.1	STUN	128 Binding Request user: vvd2:
89	39.675096400	192.168.56.1	192.168.56.1	STUN	96 Binding Success Response XO
90	39.999941000	172.22.192.1	172.22.192.1	UDP	86 62893 → 62896 Len=54
91	40.263040300	172.22.192.1	172.22.192.1	UDP	82 62896 → 62893 Len=50
92	40.735259700	172.22.192.1	172.22.192.1	STUN	128 Binding Request user: vvd2:
93	40.735536100	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
94	41.275890100	10.100.210.30	10.100.210.30	STUN	128 Binding Request user: vvd2:
95	41.276170100	10.100.210.30	10.100.210.30	STUN	96 Binding Success Response XO
96	41.543484800	172.22.192.1	172.22.192.1	STUN	132 Binding Request user: fHhb:
97	41.543958400	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
98	42.075419500	192.168.56.1	192.168.56.1	STUN	128 Binding Request user: fHhb:
99	42.075715900	192.168.56.1	192.168.56.1	STUN	96 Binding Success Response XO


```

Frame 81: Packet, 61 bytes on wire (488 bits), 61 bytes captured (488 bits) on interface \Device\NPF_{...} Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
0100 .... = Version: 4
... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 57
Identification: 0xc8a8 (56488)
0000 .... = Flags: 0x0
... 0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: UDP (17)
Header Checksum: 0x6009 [validation disabled]
[Header checksum status: Unverified]
Source Address: 127.0.0.1
Destination Address: 127.0.0.1
[Stream index: 3]
User Datagram Protocol, Src Port: 49825, Dst Port: 5005
Data (29 bytes)

```

Fig. 15. Encabezado IPv4 del datagrama local, incluido su valor TTL.

2204	115.715325100	172.22.192.1	172.22.192.1	UDP	82 62896 → 62893 Len=50
------	---------------	--------------	--------------	-----	-------------------------


```

Frame 2204: Packet, 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface \Device\NPF_{...} Null/Loopback
Internet Protocol Version 4, Src: 172.22.192.1, Dst: 172.22.192.1
User Datagram Protocol, Src Port: 62896, Dst Port: 62893
Source Port: 62896
Destination Port: 62893
Length: 58
Checksum: 0x9d35 [unverified]
[Checksum Status: Unverified]
[Stream index: 0]
[Stream Packet Number: 232]
[Timestamps]
UDP payload (50 bytes)
Data (50 bytes)
Data: 2f9b6a02d638a9e8ed75bec60f4a37f35ffc6c2c2b5a563d4ed8bcc0bf076438123536bf4a2a9e2f18fe0bad4
[Length: 50]

```

Fig. 16. Campos del datagrama UDP enviado mediante loopback.

6) Examine the Time to Live (TTL) field in the IP header of the UDP packet. Why is the TTL value what it is, and how would it change in a multi-hop real network scenario?

El ttl obtenido en este paquete es el 128. Este es el valor predeterminado que tiene Windows cuando un paquete IP se crea. Este valor se ha mantenido debido a que la dirección de origen y destino son las mismas, así que al no haber viajado a un router externo, el valor se mantiene como el mismo. Por cada salto que realizaría en un escenario real en la web el TTL se reduce en 1. Si es que este valor llega a 0, la web nos botará el mensaje de Time limit Exceeded y el router descarta el paquete. (Fig. 17)

7) Explore how Wireshark differentiates between application-layer protocols when using UDP (e.g., DNS, DHCP). Is your custom message interpreted as a known protocol? Why or why not?

81	35.959854900	127.0.0.1	127.0.0.1	UDP	61 49825 → 5005 Len=29
82	36.198536300	172.22.192.1	172.22.192.1	STUN	132 Binding Request user: fHhb:
83	36.199050700	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
84	38.056605800	172.22.192.1	172.22.192.1	STUN	128 Binding Request user: vvd2:
85	38.057066500	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
86	38.874350000	172.22.192.1	172.22.192.1	STUN	132 Binding Request user: fHhb:
87	38.874727400	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
88	39.674770500	192.168.56.1	192.168.56.1	STUN	128 Binding Request user: vvd2:
89	39.675096400	192.168.56.1	192.168.56.1	STUN	96 Binding Success Response XO
90	39.999941000	172.22.192.1	172.22.192.1	UDP	86 62893 → 62896 Len=54
91	40.263040300	172.22.192.1	172.22.192.1	UDP	82 62896 → 62893 Len=50
92	40.735259700	172.22.192.1	172.22.192.1	STUN	128 Binding Request user: vvd2:
93	40.735536100	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
94	41.275890100	10.100.210.30	10.100.210.30	STUN	128 Binding Request user: vvd2:
95	41.276170100	10.100.210.30	10.100.210.30	STUN	96 Binding Success Response XO
96	41.543484800	172.22.192.1	172.22.192.1	STUN	132 Binding Request user: fHhb:
97	41.543958400	172.22.192.1	172.22.192.1	STUN	96 Binding Success Response XO
98	42.075419500	192.168.56.1	192.168.56.1	STUN	128 Binding Request user: fHhb:
99	42.075715900	192.168.56.1	192.168.56.1	STUN	96 Binding Success Response XO


```

Frame 81: Packet, 61 bytes on wire (488 bits), 61 bytes captured (488 bits) on interface \Device\NPF_{...} Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
0100 .... = Version: 4
... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 57
Identification: 0xc8a8 (56488)
0000 .... = Flags: 0x0
... 0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: UDP (17)
Header Checksum: 0x6009 [validation disabled]
[Header checksum status: Unverified]
Source Address: 127.0.0.1
Destination Address: 127.0.0.1
[Stream index: 3]
User Datagram Protocol, Src Port: 49825, Dst Port: 5005
Data (29 bytes)

```

Fig. 17. Encabezado IPv4 del paquete UDP capturado en la interfaz de loopback.

Wireshark revisa cuales son los puerto de origen y destino y verifica con cual coincide (53 para DNS, 67/68 DHCP). Si el puerto es desconocido, wireshark inspecciona en los primeros bytes patrones que se pueden encontrar en ciertos protocolos. Ahora mismo nuestro mensaje no lo esta aceptando como un puerto conocido porque el puerto 5005 no es un puerto estandar, ahora mismo lo esta tomando como si fuera data en formato hexadecimal o texto.

8) Change the destination IP in the sender script to a non-routable or invalid IP (e.g., 192.0.2.1) and rerun the test. What happens in the capture, and why is there a difference?

Al estar intentando enviar a una direccion invalida, no lo va a encontrar en el loop, así que por eso tratara de encontrarlo en Wi-Fi y así fue. (Fig. 18, Fig. 19, Fig. 20)

D. Checkpoint 4: Python UDP Implementation (P2P)

Permitiendo el trafico por puerto UDP 5005 (Fig. 21).

1) Capture the traffic using wireshark.

2) Show the IP addresses of both laptops used.

- Laptop A receptora: (Fig. 22).
- Laptop B emisora: (Fig. 23).

3) Provide evidence of successful message transmission.

- Emisor: (Fig. 24).
- Receptor: (Fig. 25).

4) Describe the message format and how it was handled on the receiver side. El mensaje recibido es UDP. Su formato contiene el source port, destination port, length y checksum, seguido del UDP payload, que contiene 19 bytes de datos. (Fig. 26).

5) Suggest one scenario where UDP would be preferred over TCP and justify.

UDP puede preferirse en un juego multijugador en tiempo real, donde los jugadores envían constantemente

The image shows a VS Code editor window with a file named `udp_sender.py` open. The script is a Python program that sends a UDP packet. It imports `socket`, `zlib`, and `struct`. It defines a `checksum_calculator` function that uses `zlib.crc32` to calculate the checksum of the data. The main part of the script sets `UDP_IP` to `"192.0.2.1"`, `UDP_PORT` to `5005`, and `MESSAGE` to `b"Hello, World!"`. It then prints these values and calculates the `data_length` as `len(MESSAGE)`.

Below the editor, the `Terminal` tab is active, showing the output of running `python udp_sender.py` in a PowerShell prompt. The output matches the print statements in the script.

```

udp_receiver.py X  udp_sender.py X
udp_sender.py > ...
1  import socket
2  import zlib
3  import struct
4
5  #checksum generation function
6  def checksum_calculator(data):
7      checksum = zlib.crc32(data)
8      return checksum
9
10 UDP_IP = "192.0.2.1"
11 UDP_PORT = 5005
12 MESSAGE = b"Hello, World!"
13
14 print("UDP target IP: %s" % UDP_IP)
15 print("UDP target port: %s" % UDP_PORT)
16 print("message: %s" % MESSAGE)
17
18
19 data_length = len(MESSAGE)
...
Problems  Output  Debug Console  Terminal  Ports
● PS D:\repos\redes\labUDP> py udp_sender.py
UDP target IP: 192.0.2.1
UDP target port: 5005
message: b'Hello, World!'
Data length: 13
Checksum: 4
Header: b'\x00\x00\x13\x8d\x00\x00\x13\x8d\x00\x00\x00\r\x00\x00\x00\x04'
○ PS D:\repos\redes\labUDP>

```

Fig. 18. Código del emisor UDP tras modificar la dirección IP de destino.

[illegible]

Fig. 19. Salida del emisor UDP después de cambiar la dirección de destino.

actualizaciones de posición y movimiento. También puede ser útil para aplicaciones interactivas como VoIP o video en vivo. Al no requerir el *three-way handshake* de TCP ni esperar confirmaciones propias del protocolo para cada datagrama, UDP permite que la aplicación priorice las actualizaciones recientes [2], [3].

En estos casos, puede ser más útil recibir la siguiente actualización que recuperar una anterior que ya perdió relevancia. TCP, en cambio, retransmite los datos perdidos y los entrega en orden. Si falta un segmento, los datos posteriores de esa conexión pueden quedar pendientes de entrega a la aplicación hasta que se recupere, lo que puede introducir pausas perceptibles [?], [3].

- 6) **Change the message content and observe how the packet length changes in Wireshark.**
El tamaño del mensaje cambia de 27 a 113. (Fig. 27).
- 7) **Change the message content specifically to an absurd size (like 12 000 bytes) and see the behaviour of the messages. Is it possible to send messages with that**

The image displays a Wireshark packet capture analysis of a UDP flood attack. The top pane shows a list of 101 UDP packets from 10.100.210.30 to 10.100.210.30. The middle pane shows the details of packet 94, which is a UDP packet with length 5005. The bottom pane shows the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
68	4.317020500	10.100.210.30	108.177.123.95	UDP	686	60786 → 443 Len=644
69	4.328312600	108.177.123.95	10.100.210.30	QUIC	240	Protected Payload (KP0)
70	4.328718900	10.100.210.30	108.177.123.95	QUIC	75	Protected Payload (KP0), DCI
71	4.339305800	108.177.123.95	10.100.210.30	QUIC	65	Protected Payload (KP0)
72	4.403344800	108.177.123.95	10.100.210.30	UDP	70	443 → 60786 Len=28
73	4.442408100	10.100.210.30	108.177.123.95	UDP	74	60786 → 443 Len=32
74	4.608845000	108.177.123.95	10.100.210.30	UDP	127	443 → 60786 Len=85
75	4.646019600	10.100.210.30	108.177.123.95	UDP	78	60786 → 443 Len=36
76	4.711682300	108.177.123.95	10.100.210.30	UDP	127	443 → 60786 Len=85
77	4.711684700	108.177.123.95	10.100.210.30	UDP	66	443 → 60786 Len=24
78	4.712080300	10.100.210.30	108.177.123.95	UDP	75	60786 → 443 Len=33
79	7.375718100	142.250.0.95	10.100.210.30	UDP	122	443 → 63236 Len=80
80	7.387896300	10.100.210.30	142.250.0.95	UDP	75	63236 → 443 Len=33
81	8.322420000	10.100.210.30	10.100.2.22	DNS	92	Standard query 0xab7 HTTFS
82	8.322705000	10.100.210.30	10.100.2.22	DNS	92	Standard query 0xab7080 A mobi
88	8.331422000	10.100.2.22	10.100.210.30	DNS	212	Standard query response 0xab7
89	8.499779700	10.100.2.22	10.100.210.30	DNS	196	Standard query response 0xab7
94	8.758970700	10.100.210.30	192.0.2.1	UDP	71	58820 → 5005 Len=29
98	9.792680800	10.100.210.30	142.251.0.113	UDP	71	61552 → 443 Len=29
99	9.918043900	10.100.210.30	142.251.0.113	UDP	71	61552 → 443 Len=29
100	9.924257200	142.251.0.113	10.100.210.30	UDP	68	443 → 61552 Len=26
101	10.036450500	142.251.0.113	10.100.210.30	UDP	69	443 → 61552 Len=27

Packet 94 details:

- Ethernet II, Src: VirtualBox__08:00:00:00:00:00, Dst: VirtualBox__08:00:00:00:00:00, Encapsulated: Cisco ff:ff:20 (00:00:e3:ff:ff:20)
- Internet Protocol Version 4, Src: 10.100.210.30, Dst: 192.0.2.1
- 0100 = Version: 4
- ...0101 = Header Length: 20 bytes (5)
- Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
- Total Length: 57
- Identification: 0x0e2c (3628)
- 000... = Flags: 0x0
- ...0 0000 0000 0000 = Fragment Offset: 0
- Time to Live: 128
- Protocol: UDP (17)
- Header Checksum: 0x8e04 (validation disabled)
- [Header checksum status: Unverified]
- Source Address: 10.100.210.30
- Destination Address: 192.0.2.1
- [Stream index: 101]
- User Datagram Protocol, Src Port: 58820, Dst Port: 5005
- Data (29 bytes)

Fig. 20. Datagrama UDP observado tras cambiar el destino fuera de loopback.

Windows Defender Firewall con seguridad avanzada

Archivo Acción Ver Ayuda

Windows Defender Firewall con...

- Reglas de entrada
- Reglas de salida
- Reglas de seguridad de con...
- Supervisión

Nombre	Grupo	Perfil	Habilitado
Python UDP 5005	Python	Todos	Sí
AacAmbientlighting		Todo	Sí
ACSETUP	Publico	Publico	Sí
ACSETUP	Publico	Publico	Sí
ACSETUP	Publico	Publico	Sí
ACSETUP	Publico	Publico	Sí
ACSETUP	Publico	Publico	Sí
ACSETUP	Publico	Publico	Sí
APCET110	Publico	Publico	Cl

Acciones

- Reglas de entrada
- Nueva regla...
- Filtrar por perfil...
- Filtrar por estado
- Filtrar por grupo
- Ver
- Actualizar
- Exportar lista...

Fig. 21. Regla del firewall que permite el tráfico entrante por el puerto UDP 5005.

payload size?

El mensaje es enviado pero no es recibido en el receptor.
(Fig. 28).

El receptor no muestra señales de recibir el mensaje, tampoco se muestra en wireshar. Como máximo se logró recibir un mensaje de 1464 bytes. (Fig. 28).

E. Checkpoint 5

- 1)
- 2)
- 3)
- 4)
- 5)
- 6)

F. Checkpoint 6

- 1)
- 2)
- 3)
- 4)
- 5)
- 6)

```

Adaptador de LAN inalámbrica Conexión de área local* 1:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

Adaptador de LAN inalámbrica Conexión de área local* 10:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

Adaptador de Ethernet VMware Network Adapter VMnet1:

    Sufijo DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . : fe80::b5b8:fd99:6932:fef0%8
    Dirección IPv4. . . . . : 192.168.152.1
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . :

Adaptador de Ethernet VMware Network Adapter VMnet7:

    Sufijo DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . : fe80::a0db:6e7b:66d2:49d8%3
    Dirección IPv4. . . . . : 192.168.20.1
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . :

Adaptador de Ethernet VMware Network Adapter VMnet8:

    Sufijo DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . : fe80::eabb:249b:430d:17c1%11
    Dirección IPv4. . . . . : 192.168.117.1
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . :

Adaptador de LAN inalámbrica Wi-Fi:

    Sufijo DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . : fe80::f389:45d2:8cdd:209d%23
    Dirección IPv4. . . . . : 10.134.165.146
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . : 10.134.165.50

Adaptador de Ethernet Conexión de red Bluetooth:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

```

Fig. 22. Configuración IP de la laptop utilizada como receptora.

```

Adaptador de LAN inalámbrica Conexión de área local* 1:

Estado de los medios. . . . . : medios desconectados
Sufijo DNS específico para la conexión. . .

Adaptador de LAN inalámbrica Conexión de área local* 2:

Estado de los medios. . . . . : medios desconectados
Sufijo DNS específico para la conexión. . .

Adaptador de Ethernet Ethernet:

Estado de los medios. . . . . : medios desconectados
Sufijo DNS específico para la conexión. . .

Adaptador de LAN inalámbrica Wi-Fi:

Sufijo DNS específico para la conexión. . .
Vínculo: dirección IPv6 local. . . : fe80::c726:b57a:b4f9:f50d%19
Dirección IPv4. . . . . : 10.134.165.66
Máscara de subred . . . . . : 255.255.255.0
Puerta de enlace predeterminada . . . . : 10.134.165.50

Adaptador de Ethernet Conexión de red Bluetooth:

```

Fig. 23. Configuración IP de la laptop utilizada como emisora.

Wireshark packet capture showing an ICMP Echo (ping) request. The packet list shows a packet from 10.134.165.146 to 10.134.165.146. The packet details show the Ethernet II, Internet Protocol Version 4, and User Datagram Protocol (ping) fields. The packet bytes show the ICMP Echo request structure.

No.	Time	Source	Destination	Protocol	Length	Info
464	2026-09-05 13:42:32.5	10.134.165.66	10.134.165.146	UDP	61	59161 → 5005 Len=19

Frame 464: Packet, 61 bytes
 Ethernet II, Src: Intel, 44:00:00:2f:27:cc, 40:00:3f:1b:b4:1, 0a:86:a5:42:9a:86
 Internet Protocol Version 4, 52:e7:19:18:8d:0b:a7:13:8d:13:8d:0b:0b
 User Datagram Protocol, Src Port: 59161, Dest Port: 5005
 Data (19 bytes):
 0000 20 1c 88 c5 a5 44 ca bd c5 f4 5a 8d 08 00 45 00 ... 0 ... Z ... E ...
 0010 00 2f 27 cc 40 00 3f 1b b4 11 0a 86 a5 42 9a 86 ... / ... ? ... B ...
 0020 a5 92 e7 19 18 8d 0b a7 13 8d 13 8d 0b 0b ...
 0030 00 16 48 45 4c 4c 4f 20 5f 4f 52 4c 44 ... -HELLO WORLD

Fig. 24. Datagrama UDP enviado entre las dos laptops por el puerto 5005.

```

1128 6e.0720b090      10.134.165.66      10.134.165.146      TCP      61 59161 > 5005 Len=0
1129 6e.0720b090      79.127.223.24      10.134.165.146      UDP      57 8886 > 59121 [PSH, ACK] Seq=5 Acks=5 Win=0 Len=3
1130 6e.072594d0      10.134.165.146     79.127.223.24      TCP      59 59121 > 8886 [PSH, ACK] Seq=5 Acks=7 Win=513 Len=3

# Frame 1128: Packet, 6 bytes on wire (488 bits), 61 bytes captured (488
#       bytes on interface) (capture length limit: 65535 bytes)
# Ethernet II, Src: Intel F4:5a:d2 (c4:b5:a4:F4:5a:d2), Dst: Intel c8:a5:
#       00:20:4f:27:c1:19 3d 04 00 3f 11 b4 10 06 02 04 06
# Internet Protocol Version 4, Src: 10.134.165.66, Dst: 10.134.165.146
# User Datagram Protocol, Src Port: 59161, Dst Port: 5005
# Data (19 bytes)
HELLO WORLD

```

Fig. 25. Paquete capturado durante el intercambio UDP entre las laptops.

```

1128 41:072678000      10.134.165.66      10.134.165.164      TCP      61 59101 → 5005 [Len=1]
1129 46:072879000      79.127.223.24      10.134.165.164      TCP      57 8886 → 59121 [PSH, ACK] Seq=4505 Win=8 Len=3
1130 46:072856000      10.134.165.164      79.127.223.24      TCP      57 59121 → 8886 [PSH, ACK] Seq=4507 Win=3 Len=3

Frame 1128: Packet, 61 bytes on wire (488 bits), 61 bytes captured (488 bits) on interface 0
Ethernet II, Src: IntelE810S40 (ca:40:10:13:14:16:54:66), Dst: IntelE810S40 (ca:40:10:13:14:16:54:16)
Internet Protocol Version 4, Src: 10.134.165.66, Dst: 10.134.165.164
User Datagram Protocol, Src Port: 59101, Dst Port: 5005
Source Port: 59101
Destination Port: 5005
Length: 27
Checksum: Baza7 [unverified]
[Checksum Status: Unverified]
[Stream index: 24]
[Stream Capture Number: 1]
+ [Timestamp]
User payload (19 bytes)
+ Data (19 bytes)

```

Fig. 26. Puertos, longitud y carga útil del datagrama UDP recibido.

G. Checkpoint 7

- 1)
- 2)
- 3)
- 4)
- 5)
- 6)

H. Checkpoint 8

- 1)
- 2)
- 3)
- 4)
- 5)
- 6)

I. Checkpoint 9: UDP Vulnerabilities: Simulating a Local DDoS (UDP Flood)

- 1) **On the Victim's Wireshark, go to Statistics ; I/O Graphs. Provide a screenshot of the graph showing the sudden spike in packets/bytes per second during the 15-second attack window.**
 - Atacantes: Fig. 30.
 - Victima: Fig. 31.
- 2) **Based on the Victim's OS Resource Monitor, did you notice a significant spike in Network bandwidth**

[illegible]

Fig. 27. Datagrama capturado después de modificar el contenido del mensaje.

[illegible]

Fig. 28. Salida del programa al intentar enviar un mensaje UDP de gran tamaño.

No.	Time	Source	Destination	Protocol	Length	Info
199	4.876849980	10.134.165.66	10.134.165.146	UDP	51	59205 → 5005 Len=9
263	13.429318000	10.134.165.66	10.134.165.146	UDP	1259	59235 → 5005 Len=1208
2304	108.605427600	10.134.165.66	10.134.165.146	UDP	1050	59193 → 5005 Len=1008
4004	168.201703800	10.134.165.66	10.134.165.146	UDP	1350	59234 → 5005 Len=1308
4279	177.534164200	10.134.165.66	10.134.165.146	UDP	1400	59201 → 5005 Len=1358
4792	202.564727800	10.134.165.66	10.134.165.146	UDP	1440	59138 → 5005 Len=1398
4886	211.282044100	10.134.165.66	10.134.165.146	UDP	1450	59191 → 5005 Len=1408
5198	235.230100700	10.134.165.66	10.134.165.146	UDP	1500	59209 → 5005 Len=1458
5416	255.177907800	10.134.165.66	10.134.165.146	UDP	1512	59212 → 5005 Len=1470
5901	270.207112400	10.134.165.66	10.134.165.146	UDP	1513	59167 → 5005 Len=1471
6239	280.015830400	10.134.165.66	10.134.165.146	UDP	1514	59236 → 5005 Len=1472

Fig. 29. Longitudes de los datagramas observados durante la prueba con mensajes de distintos tamaños.

```

Problems Output Debug Console Terminal Ports
renat@PS: MITNWS4 ~ /OneDrive/Documents/2026/2026-2/redes/code/lab2
$ python udp_flood.py
Enviando UDP a 172.20.10.3:5005 durante 15 segundos.
Puedes detenerlo antes con Ctrl + C.
Paquetes enviados: 8952
Tiempo transcurrido: 15.00 segundos

sofaku@MSI:/mnt/c/Users/sofia/Desktop/Documents/2026/2026-2/redes$ python3 udp_flood.py
Enviando UDP a 172.20.10.3:5005 durante 15 segundos.
Puedes detenerlo antes con Ctrl + C.
Paquetes enviados: 13355
Tiempo transcurrido: 15.00 segundos

PS C:\Users\Admin\Documents\redes\lab4> python3 lab7.py
Enviando UDP a 172.20.10.3:5005 durante 15 segundos.
Puedes detenerlo antes con Ctrl + C.
Paquetes enviados: 7667
Tiempo transcurrido: 15.00 segundos

PS D:\repos\redes\labUDP> py udp_flood.py
Enviando UDP a 172.20.10.3:5005 durante 15 segundos.
Puedes detenerlo antes con Ctrl + C.
Paquetes enviados: 9295
Tiempo transcurrido: 15.00 segundos
PS D:\repos\redes\labUDP>

```

Fig. 30.

usage or CPU utilization? Explain why an enterprise server might crash if thousands of external computers did this simultaneously.

Sí, durante la prueba se observó un aumento en el tráfico de red, que alcanzó aproximadamente 18 Mb/s durante el ataque. No se observó un incremento significativo de CPU ya que se observa un uso del 2% casi constante (Fig. 31). Si miles de computadoras enviaran paquetes simultáneamente a un servidor empresarial, podrían saturar su ancho de banda y agotar sus recursos

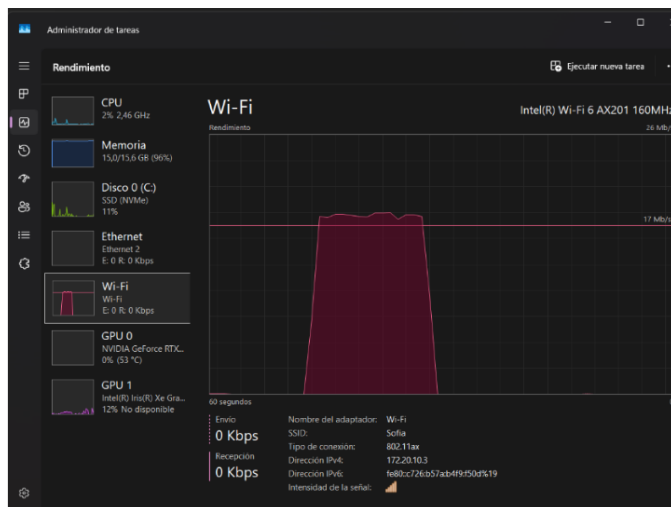


Fig. 31.

de procesamiento. Esto podría causar lentitud, pérdida de paquetes y que el servidor dejara de responder a los usuarios legítimos, aunque no necesariamente se apagara.

- 3) **Why is a "Flood" attack fundamentally easier to execute using UDP rather than TCP? (Hint: Think about what TCP requires before sending data, and what UDP ignores).**

Flood attack es fundamentalmente más fácil de ejecutar usando UDP porque UDP no hace handshake antes de enviar datos. Esto permite enviar muchos paquetes seguidos sin esperar una respuesta del receptor. En cambio, TCP requiere establecer una conexión para transferir datos y cuenta con mecanismos de control de flujo y congestión que regulan el envío.

- 4) **One common tactic in real-world UDP attacks is "IP Spoofing," where the attacker forges the Source IP address. Why is it relatively easy to spoof an IP address in a UDP packet, but extremely difficult to do so in a TCP connection?**

Esto se debe principalmente a la seguridad entre ambos. UDP ha sido diseñado con los criterios de simplicidad y velocidad, por eso no cuenta con handshake ni con verificación. Por otra parte, el TCP ha sido designado con un enfoque de seguridad. Este a comparación del protocolo UDP, cuenta con three-way-handshake (SYN, SYN-ACK, ACK).

- 5) **If you were the network administrator for the Victim machine, look at the captured flood packets. Is there any legitimate pattern you could use to block this traffic using a Firewall rule? How would the firewall differentiate this flood from legitimate UDP traffic (like a DNS response or a Skype call)?**

Sí se puede identificar el patrón anormal por la gran cantidad de paquetes UDP enviados en muy poco tiempo. El firewall podría bloquear o limitar el tráfico cuando una misma IP supera cierta cantidad de paquetes por segundo. También podría considerar el puerto de destino y el tamaño de los paquetes para detectar el tráfico sospechoso. (Fig. 32)

- 6) **Research what a "UDP Amplification Attack" is (e.g., DNS or NTP amplification). Briefly explain how an attacker uses a small UDP request to trick a third-party server into sending a massive UDP response to the victim**

III. CONCLUSION

CONCLUSION

CONCLUSIONES

A. Primera experiencia

En primer lugar, se capturó el tráfico UDP general del navegador y se identificaron los campos principales de cada paquete, lo que nos permitió observar el "unverified" en el checksum. Esto corresponde a la delegación del cálculo a la tarjeta de red: el hardware realiza la verificación antes de entregar el paquete a Wireshark, por lo que este no puede

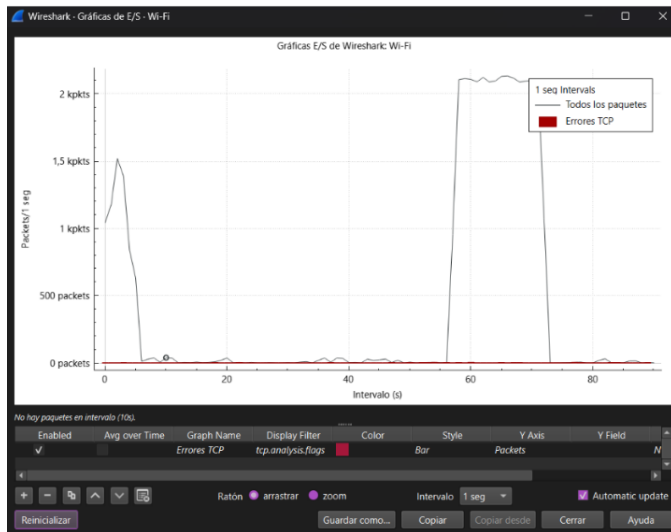


Fig. 32.

confirmar si es correcto. Asimismo, una de las diferencias entre TCP y UDP radica en las banderas ACK y PSH, propias del mecanismo de control de TCP y ausentes en UDP.

B. Segunda experiencia

En este caso, al usar nslookup para generar tráfico DNS sobre UDP, identificamos que el servidor DNS siempre responde por el puerto 53, mientras que el host usa un puerto temporal alto para cada consulta. Evidenciamos que las respuestas pesan más que las solicitudes por incluir registros adicionales (IP, CNAME). Concluimos que UDP es ideal para DNS porque su ausencia de handshake reduce la latencia en consultas breves y frecuentes..

C. Tercera experiencia

Al realizar envíos de datagramas UDP con Python, donde el header recibido coincidía con el generado por el emisor confirmamos que no hay alteración de datos en el trayecto. Además, notamos que el envío de UDP no realiza handshake y al cambiar el tamaño del mensaje enviado concluimos que existe un límite práctico de tamaño (aprox. 1464 bytes).

D. Quinta experiencia

Enviando un mensaje en broadcast notamos que los tres receptores lo capturaron simultáneamente con la misma MAC destino e IP de broadcast de la subred. Esta forma de transmisión es exclusiva de UDP, TCP no puede replicarla al requerir una conexión punto a punto con cada receptor, por lo que concluimos que esta capacidad es la razón por la que protocolos como DHCP y SSDP dependen del broadcast UDP para funcionar sin configuración manual previa.

E. Séptima experiencia

Mediante NetCat notamos que no había tráfico previo al envío del mensaje, confirmando la ausencia de handshake en UDP. Esta simplicidad también se refleja en su header

de menor tamaño (8 bytes), lo cual lo hace preferible para aplicaciones que priorizan la velocidad. Sin embargo, al enviar a un puerto cerrado se genera un ICMP "Port Unreachable", lo que evidencia que UDP sigue sin ser confiable, pues el sistema operativo no retransmite automáticamente el paquete perdido.

REFERENCES

- [1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. PEARSON, 2021.
- [2] J. Postel, "User datagram protocol," RFC Editor, Request for Comments RFC 768, 1980. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc768>
- [3] W. Eddy, "Transmission control protocol (tcp)," RFC Editor, RFC 9293, Aug. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9293>
- [4] P. Mockapetris, "Domain names—concepts and facilities," RFC Editor, Request for Comments RFC 1034, 1987. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1034>
- [5] —, "Domain names—implementation and specification," RFC Editor, RFC 1035, 1987. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1035>
- [6] R. Bonica, F. Baker, G. Huston, R. Hinden, O. Troan, and F. Gont, "Ip fragmentation considered fragile," RFC Editor, RFC 8900, Sep. 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8900>
- [7] W. Foundation, "Wireshark user's guide." [Online]. Available: https://www.wireshark.org/docs/wsug_html_chunked/
- [8] J. Postel, "Internet control message protocol," RFC Editor, RFC 792, Sep. 1981. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc792>
- [9] J. Mogul, "Broadcasting internet datagrams," RFC Editor, RFC 919, Oct. 1984. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc919>
- [10] R. Droms, "Dynamic host configuration protocol," RFC Editor, RFC 2131, Mar. 1997. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2131>
- [11] E. Gelenbe and M. Nasereddin, "Protecting IoT servers against flood attacks with the quasi deterministic transmission policy," in *2023 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2023, pp. 379–386.
- [12] Nmap Project, "Ncat Users' Guide," accessed: 2026-09-25. [Online]. Available: <https://nmap.org/ncat/guide/>
- [13] A. M. Al-Jubari, M. O. Ali, and A. K. Al-Mashhadani, "Performance comparison between tcp and udp protocols in different simulation scenarios," *Int. J. Eng. Technol.*, vol. 7, no. 4, pp. 172–176, 2018.
- [14] NetBIOS Working Group, "Protocol standard for a NetBIOS service on a TCP/UDP transport: Detailed specifications," RFC Editor, RFC 1002, 1987. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1002>